Learning the Rank of an Elliptic Curve

Lecture notes written while building `elliptic-curve-intelligence-lab`

Dylan Costa

Contents

0 The problem: which rank are we predicting? **1**

1 Data: what a row means, and where it came from **4**

2 What the model is allowed to see **7**

3 Evaluation when examples aren't independent **10**

4 Baselines and logistic regression **11**

5 Gradient boosting, and what each feature group adds **14**

6 A small neural network, and whether to trust the probabilities **17**

Solutions to exercises **19**

0 The problem: which rank are we predicting?

We want a function f that takes an elliptic curve $E/\mathbb{Q}$, given by its Weierstrass coefficients, and returns the probability that its rank is positive (Task 1) or, among curves of even rank, that its rank is 2 rather than 0 (Task 2). Computing the rank is a hard problem with no known algorithm guaranteed to terminate. Predicting it is a statistical question: given a few million curves whose ranks are known, which cheaply computable features of a curve carry information about its rank, and does that information survive on curves larger than the ones we trained on? This section pins down what "rank" means, which rank our labels are, and why Task 2, not Task 1, is the honest version of the question.

Curves and their rational points. An elliptic curve over $\mathbb{Q}$ is a smooth projective curve given by a Weierstrass equation

$$E:\ y^2 + a_1xy + a_3y = x^3 + a_2x^2 + a_4x + a_6, \qquad a_i \in \mathbb{Z},$$

with nonzero discriminant Δ. Its rational points form an abelian group, and Mordell's theorem (1922) says this group is finitely generated:

$$E(\mathbb{Q}) \ \cong\ E(\mathbb{Q})_{\text{tors}} \oplus \mathbb{Z}^r, \qquad r = \operatorname{rk} E(\mathbb{Q}).$$

The torsion part is well understood: by Mazur (1977) it is one of exactly 15 groups, and it is easy to compute. The *algebraic rank* r is the mystery. A curve has rank 0 when it has only finitely many rational points, and positive rank when it has infinitely many. Our dataset stores each curve by its label and coefficients, for example `11.a1|[0,-1,1,-7820,-263580]|0|[]|11|1`: label, $[a_1, a_2, a_3, a_4, a_6]$, rank, torsion structure, conductor, and the analytic order of the Shafarevich–Tate group Sha.

Counting points mod p, and the L-function. Reducing the equation mod a prime p gives a curve over $\mathbb{F}_p$ with some number of points. Write

$$a_p \;=\; p + 1 - \#E(\mathbb{F}_p), \qquad |a_p| \le 2\sqrt{p} \quad \text{(Hasse, 1933) for } p \nmid N.$$

Here N is the *conductor*, a product of the primes where E has bad reduction (with exponents that record how bad). The a_p assemble into the L-function

$$L(E,s) = \prod_{p \nmid N} \bigl(1 - a_p p^{-s} + p^{1-2s}\bigr)^{-1} \prod_{p \mid N} \bigl(1 - a_p p^{-s}\bigr)^{-1},$$

which converges for $\operatorname{Re} s > 3/2$. By the modularity theorem (Wiles; Taylor–Wiles; Breuil–Conrad–Diamond–Taylor, 2001), $L(E,s)$ extends to an entire function, and $\Lambda(E,s) = N^{s/2}(2\pi)^{-s}\Gamma(s)L(E,s)$ satisfies the functional equation

$$\Lambda(E,s) \;=\; w\,\Lambda(E,2-s), \qquad w \in \{\pm 1\}. \tag{1}$$

The sign w is the *root number*. The *analytic rank* is $r_{\mathrm{an}} = \operatorname{ord}_{s=1} L(E,s)$, the order of vanishing at the center of symmetry.

Conjecture 0.1 (Birch and Swinnerton-Dyer)**.** *$r_{\mathrm{an}} = r$. Moreover the leading Taylor coefficient is*

$$\frac{L^{(r)}(E,1)}{r!} \;=\; \frac{\Omega_E \cdot \mathrm{Reg}(E) \cdot \prod_{p \mid N} c_p \cdot \#\,\mathrm{Sha}(E)}{\#E(\mathbb{Q})^2_{\mathrm{tors}}},$$

with Ω_E the real period, Reg *the regulator of a basis of $E(\mathbb{Q})/$torsion, and c_p the Tamagawa numbers.*

The conjecture grew out of numerical experiments in the 1960s with exactly the quantity we will use: Birch and Swinnerton-Dyer observed that $\prod_{p \le X} \#E(\mathbb{F}_p)/p$ seems to grow like $C\,(\log X)^r$. Curves of higher rank have, on average, *more* points mod p, so their a_p lean negative. That is the signal a model can hope to find.

Which rank are our labels? For small ranks, the two ranks are known to agree, and this is what makes the labels trustworthy.

Theorem 0.2 (Gross–Zagier 1986, Kolyvagin 1989, plus modularity)**.** *If $r_{\mathrm{an}} \le 1$, then $r = r_{\mathrm{an}}$ and* Sha(E) *is finite.*

Proposition 0.3. *Suppose $r \le 3$ is proven for E (by exhibiting r independent points and an upper bound from descent), that $L^{(r)}(E,1) \neq 0$ has been certified numerically, and that the root number satisfies $w = (-1)^r$. Then $r_{\mathrm{an}} = r$.*

Proof. Since $L^{(r)}(E,1) \neq 0$, $r_{\mathrm{an}} \le r$. If $r \le 1$, then $r_{\mathrm{an}} \le 1$ and the theorem gives $r_{\mathrm{an}} = r$. If $r \ge 2$, then $r_{\mathrm{an}} \le 1$ is impossible, because the theorem would force $r = r_{\mathrm{an}} \le 1$. So $2 \le r_{\mathrm{an}} \le r$. For $r = 2$ we are done. For $r = 3$, Exercise 0.1 gives $w = (-1)^{r_{\mathrm{an}}}$, so r_{an} is odd, hence 3. □

For rank 4 the argument breaks down: $r_{\mathrm{an}} \in \{2, 4\}$, and nothing known rules out 2. That is one reason Task 2 stops at rank 2. Section 1 checks how the dataset's ranks were obtained, since the curves above conductor 500,000 come from a different source than the complete tables below it.

The parity shortcut, and why Task 1 is not enough. The root number is not a mysterious analytic quantity. It factors into local signs, $w = -\prod_{p|N} w_p$, and each w_p is determined by the reduction of E at p. For multiplicative reduction, $w_p = -a_p$, where $a_p = +1$ (split) or -1 (non-split). Additive reduction is more involved but still a finite local computation. So w is a cheap, deterministic function of the coefficients, and by Exercise 0.1,

$$w = (-1)^{r_{\mathrm{an}}}.$$

Every curve with $w = -1$ has odd, hence positive, rank. A flexible model trained on Task 1 can therefore get a large part of the way by rediscovering the parity of the rank from the bad primes, which is a theorem rather than a discovery. Dropping a "root number" column does not fix this, because the model can rebuild w from features we do allow. The fix is to change the question so that parity carries no information:

task	curves	label
Task 1	all	$r > 0$
Task 2	$w = +1$, i.e. $r \in \{0, 2\}$	$r = 2$

In Task 2, w is constant, so any skill a model shows must come from somewhere else, and the Birch–Swinnerton-Dyer heuristic says where to look: the a_p. Because w is known and cheap, filtering the curves in the code is a simple rank filter:

From src/eclab/tasks.py

```
TASK2_RANKS = (0, 2)

def task2_mask(rank) -> np.ndarray:
    """Which curves belong to Task 2: those of rank 0 or 2."""
    return np.isin(np.asarray(rank), TASK2_RANKS)
```

We keep Task 1 anyway, for two reasons: it is the question most people would ask first, and comparing a model's score on Task 1 with its score on Task 2 measures how much of its Task 1 skill was just parity.

What the model may not see. Several quantities are computed *from* the rank, or from the L-function at $s = 1$, and would turn the experiment into a lookup. Section 2 audits every feature; the main exclusions are already clear from the BSD formula:

- $L(E, 1)$ and the leading coefficient: $L(E, 1) \neq 0$ if and only if $r_{\mathrm{an}} = 0$.
- The regulator and generators of $E(\mathbb{Q})$: they exist only once the rank is known.
- The analytic order of Sha (the last column of our data): it is computed from the BSD formula, which needs the rank and the leading coefficient.

Torsion, the conductor, the discriminant, Tamagawa numbers and the a_p are allowed. They are properties of the curve or its reduction mod p, not of the rank.

What success means here. A high score is not the goal. Ranks are already known for every curve in the dataset, so the point is the scientific question: how much of the rank is visible in the a_p for small p, and does what a model learns at small conductor carry over to large conductor? A careful "not much, and here is why" is a result. This question is not new: He, Lee, Oliver and Pozdnyakov trained classifiers on a_p data from the LMFDB and found, by looking at what the classifiers used, a new phenomenon in the averages of a_p by rank ("murmurations", 2022).

Section 7 places our results relative to theirs. What these notes add is the parity-free Task 2, the out-of-distribution test, and the leakage audit.

Roadmap. 1 data and validation · 2 features and the leakage audit · 3 grouped and out-of-distribution splits · 4 baselines · 5 gradient boosting and ablation · 6 a small neural network and calibration · 7 larger conductors and murmurations · 8 elliptic-curve signatures in practice · 9 post-quantum signatures · 10 reproducibility · 11 talking about it.

How the repo maps to these notes. The package is `src/eclab/`, tests are in `tests/`, and every number and figure from Section 1 on is produced by `scripts/figs_stepNN.py` and pulled in with `\input`, so the notes cannot silently disagree with the code.

Exercises.

Exercise 0.1. Expand $\Lambda(E, s)$ in a Taylor series around $s = 1$ and use (1) to show that $w = (-1)^{r_{\mathrm{an}}}$. (Check first that the factor $N^{s/2}(2\pi)^{-s}\Gamma(s)$ does not vanish at $s = 1$.)

Exercise 0.2. Suppose E is semistable, so every bad prime is multiplicative and N is squarefree. Show that $w = (-1)^{\omega(N)+1} \prod_{p|N} a_p$, where $\omega(N)$ is the number of primes dividing N. Then check the formula on `11.a` (rank 0, split at 11) and `37.a` (rank 1, non-split at 37).

Exercise 0.3. In Task 2 let π be the fraction of rank-2 curves. Show that the best constant predictor has accuracy $\max(\pi, 1-\pi)$ and log-loss $H(\pi) = -\pi \log \pi - (1-\pi)\log(1-\pi)$. Explain why accuracy alone makes a useless model look good when π is small, and why we report log-loss and PR-AUC as well.

Interview prep. *In 60 seconds:* "The rank of an elliptic curve is the number of independent infinite-order rational points. The obvious ML task, zero versus positive rank, is partly solved by parity: the root number is a cheap local computation and equals (-1) to the rank. So I made the headline task rank 0 versus rank 2 among curves with root number $+1$, where parity carries no signal, and tested whether what the model learns transfers to larger conductors." *A skeptic asks:* "Aren't the ranks just conjectural?" Answer with Proposition 0.3: for ranks up to 3 with proven algebraic rank, analytic and algebraic rank provably agree; that is why the tasks stop at rank 2 and leave rank 4 out.

1 Data: what a row means, and where it came from

Every claim we make later is a claim about this table, so before fitting anything we check that it says what we think it says. Step 1 parses 3,824,372 curves into a typed table (`data/curves.parquet`), runs 10 consistency checks, and asks where the curves came from. That last question turns out to matter most: the dataset is two datasets glued together.

A row. Each line of the source file is `label|[a1,a2,a3,a4,a6]|rank|[torsion]|conductor|sha`. The label `389.a1` reads as conductor 389, isogeny class `a`, first curve in the class. The coefficients are those of the *reduced global minimal model*: minimal at every prime, with $a_1, a_3 \in \{0, 1\}$ and $a_2 \in \{-1, 0, 1\}$. Every curve over $\mathbb{Q}$ has exactly one such model, so the coefficients identify the curve. The other two coefficients can be enormous: 1,228 curves have an $|a_4|$ or $|a_6|$ of at least 2^{63}, too large for a 64-bit integer. So the table stores a_4, a_6 as decimal strings and converts them to exact Python integers when needed. Storing them as floats would round them and silently change the curve.

Checks that can actually fail. A validation step is only useful if it catches real errors, so every check comes with a test that corrupts one row and confirms the check notices (`tests/test_data.py`). The strongest check uses the fact that for a minimal model, the primes of bad reduction are exactly the primes dividing the discriminant Δ. So Δ with the conductor's primes divided out must be ± 1:

From `src/eclab/data/validate.py`

```
    delta = discriminant(a1, a2, a3, a4, a6)
    if delta == 0:
        return "singular"
    for p in conductor_factors:
        if delta % p:
            return "bad_prime_not_in_discriminant"
        while delta % p == 0:
            delta //= p
    if abs(delta) != 1:
        return "discriminant_has_good_prime"
```

This one check catches a mistyped coefficient, a non-minimal model, and a conductor that belongs to a different curve. The other checks are that labels are unique and agree with the conductor, the model is reduced, the torsion group is one of Mazur's 15, the analytic Sha is a positive square (it must be, if Sha is finite, by the Cassels–Tate pairing), the exponent of each p in N is at most 8, 5 or 2 for $p = 2$, 3 or ≥ 5, and the rank is constant on each isogeny class. All 10 checks pass on all 3,824,372 curves (0 failures), which takes about a minute.

Remark 1.1 (a test that was wrong, not the code). The first test run asserted $\Delta = -11^{10}$ for `11.a1`, from memory of the curve $X_0(11)$. The code said -11. The code was right: the LMFDB numbers curves in a class by its own rule, so `11.a1` is Cremona's `11a2`, while $X_0(11)$ is `11.a2`. When a test and the code disagree, check the test's source of truth too.

Isogeny classes, and why they matter for splitting. The 3,824,372 curves fall into 2,917,287 isogeny classes. Most classes (76.9%) have a single curve; the largest has 8. Isogenous curves are different curves, but for our purposes they are copies of each other. They have the same rank (Exercise 1.1), the same conductor, and the same number of points mod every good prime, so the same a_p (Exercise 1.2). If `37.b1` lands in training and `37.b2` in testing, the model is tested on a curve whose a_p and label it has already seen. Section 3 therefore splits by isogeny class, never by curve.

Two datasets glued together. The conductors run up to 299,996,953, but not evenly:

- 3,064,705 curves have $N \leq 500{,}000$. This is Cremona's table, which lists *every* curve over $\mathbb{Q}$ of conductor below 500,000.
- 759,667 curves have $N > 500{,}000$. Every one of them has either a prime conductor (727,585) or a 7-smooth conductor, meaning no prime factor above 7 (32,082). There is not a single curve above 500,000 with any other conductor.

These come from separate searches for curves of prime conductor and of 7-smooth conductor. Our file does not record how complete those searches are, or how their ranks were obtained; that is a question for the LMFDB's source notes before we lean on those ranks.

My first guess, written into the project plan before looking, was that the large-conductor curves were a sample with small discriminants. The data said otherwise, and it changes the story of the out-of-distribution test completely. Figure 1 compares the rank mix of four groups. Above 500,000, rank 2 makes up far more of the prime-conductor curves than of the complete range. But the right

comparison is with prime-conductor curves *below* 500,000 (only 5,525 of them), and those look just like the large ones. The shift is not "larger conductor"; it is "prime conductor".

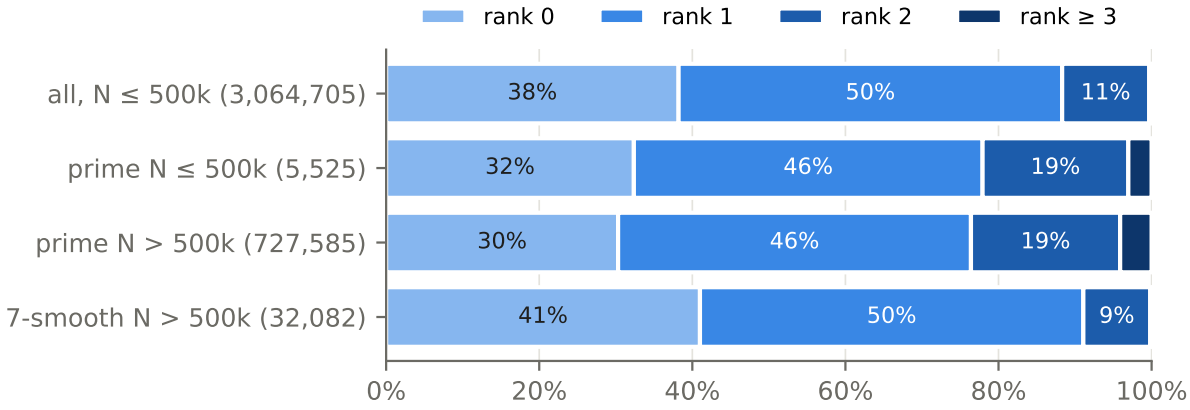


Figure 1: Rank mix by group, with the number of curves in each. Prime-conductor curves have the same mix below and above 500,000; the complete range below 500,000 is different because most of its conductors are composite.

The rank mix drifts with the conductor. Even inside the complete range the mix is not stable (Figure 2). Between the first and last bins of 25,000 conductors, the share of rank 2 rises from 5.6% to 12.8%, and rank 0 falls from 43.6% to 37.1%. Conjecturally, half of all curves have rank 0 and half rank 1, so this excess of rank 2 is expected to fade at much larger conductors, too slowly to see here. Bektemirov, Mazur, Stein and Watkins discuss this tension between tables and conjecture (*Average ranks of elliptic curves*, Bull. AMS, 2007). Two consequences for us: the conductor alone carries information about the label, and any test on larger conductors faces a different class balance, a *label shift*, even before the features change.

What this means for Task 2. Task 2 keeps the 1,897,801 curves of rank 0 or 2. Overall 26.0% of them have rank 2, but that rate is 22.9% in the complete range, 37.0% for prime conductors below 500,000, and 39.1% for prime conductors above it. A model trained on the complete range and tested on the large prime-conductor curves would face both a different kind of conductor and a different base rate. So Section 3 will run two tests: a clean one, prime conductors below 500,000 against prime conductors above it, and the harder one across the boundary.

A first look at leakage. The last column is the analytic order of Sha. It exceeds 1 for 20.4% of rank-0 curves but only 0.1% of rank-2 curves. The column is computed from the BSD formula, which needs the rank and the leading coefficient. And empirically, as in Delaunay's heuristics for Sha, nontrivial Sha is much rarer on curves of higher rank. A model given this column would learn "Sha > 1 means rank 0" and look clever. It stays out of every feature set, as Section 0 promised.

What this step cannot tell us. The checks confirm that each row describes a real, minimal curve with the stated conductor. They cannot confirm the ranks: those come from descents and L-function computations that we are trusting, not redoing. Section 2 begins computing things we *can* check ourselves: the a_p, by counting points.

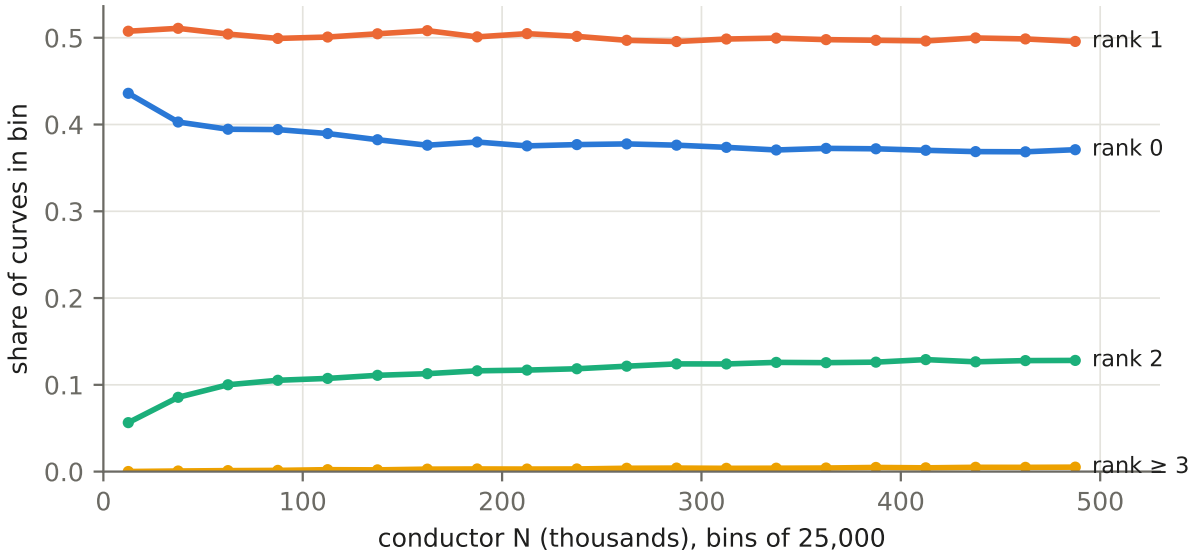


Figure 2: Share of each rank among curves of conductor $N \le 500{,}000$, in bins of 25,000.

Exercises.

Exercise 1.1. Let $\varphi : E \to E'$ be an isogeny over $\mathbb{Q}$. Show that $\operatorname{rk} E(\mathbb{Q}) = \operatorname{rk} E'(\mathbb{Q})$. (Use the dual isogeny $\hat{\varphi}$ with $\hat{\varphi} \circ \varphi = [\deg \varphi]$, and that $\ker \varphi$ is finite.)

Exercise 1.2. Show that isogenous curves have $\#E(\mathbb{F}_p) = \#E'(\mathbb{F}_p)$ for every good prime p, so their a_p agree. Then suppose we split curves (not classes) at random, 80% to training. Using the class sizes, estimate what fraction of test curves have an isogenous twin in training. Why is that fraction an underestimate of the damage when the model is very flexible?

Exercise 1.3. A model is calibrated on a population where rank 2 has base rate π, and then used on one with base rate π'. Assume pure *label shift*: the distribution of features given the rank is the same in both. Show that the correct probability on the new population is obtained by multiplying the predicted odds by $\frac{\pi'/(1-\pi')}{\pi/(1-\pi)}$. Evaluate the factor for the complete range versus large prime conductors in Task 2.

Interview prep. *"How did you validate the data?"* "Ten checks, each with a test that corrupts a row to prove it can fail. The strongest uses the fact that for a minimal model the bad primes are exactly the primes of the discriminant, which catches typos, non-minimal models and wrong conductors in one go. Then I asked where the data came from: everything above conductor 500,000 has prime or 7-smooth conductor, so the 'generalize to larger conductors' test is really a test on a different family. I compare like with like, prime conductors below 500,000 against prime conductors above it."

2 What the model is allowed to see

A feature is allowed if it is a cheap, checkable property of the curve that does not encode the rank. Step 2 computes two groups for every curve (`src/eclab/features.py`), keeps an explicit list of everything excluded, and checks the a_p against several independent facts.

Invariants (15 features). From the conductor: $\log N$, the number of bad primes, whether N is prime or squarefree, and the exponents of 2 and 3. From the minimal discriminant: $\log|\Delta|$, its sign (whether $E(\mathbb{R})$ has one or two components) and the Szpiro ratio $\log|\Delta|/\log N$. Also $\log(1+|j|)$, the torsion order, $\#E(\mathbb{Q})[2]$, and the number of primes of split multiplicative, non-split multiplicative and additive reduction. Those last three deserve a warning: together with the additive local signs they determine the root number. So they let a model learn parity on Task 1, as Section 0 predicted. On Task 2 parity is constant, so they cannot help there in that way.

Frobenius traces (25 features). $a_p = p + 1 - \#E(\mathbb{F}_p)$ for every prime $p < 100$, including bad primes, where the same count gives $a_p = 1$, -1 or 0 for split, non-split and additive reduction. For odd p, completing the square turns $y^2 + (a_1x + a_3)y = x^3 + \dots$ into $Y^2 = g(x)$ with $g(x) = 4x^3 + b_2x^2 + 2b_4x + b_6$. Each x then contributes $1 + \chi(g(x))$ points, where χ is the Legendre symbol, so

$$a_p \;=\; -\sum_{x\in\mathbb{F}_p} \chi(g(x)).$$

This turns point counting into one table lookup per (x, curve), done for a block of 200,000 curves at a time:

From src/eclab/features.py

```
g = (4 * x3 + b2[sl, None] * x2 + (2 * b4[sl, None] % p) * x + b6[sl, None]) % p
out[sl] = -chi[g].sum(axis=1, dtype=np.int64)
```

The b_i are first reduced mod p from exact Python integers, since a_4, a_6 can exceed 64 bits. All 3,824,372 curves take under four minutes. For bad primes $p \geq 100$, the reduction type uses the criterion $a_p = \left(\frac{-c_6}{p}\right)$ for multiplicative p.

Checking numbers we cannot look up. PARI does not build on this Windows machine, so instead of comparing against another program we check facts the a_p must satisfy:

- a brute-force count over all $(x, y) \in \mathbb{F}_p^2$ agrees on the test sample, and so do the q-expansions of the forms of level 11 and 37;
- the Hasse bound holds everywhere: the largest $|a_p|/2\sqrt{p}$ is 0.995;
- isogenous curves have equal a_p (Exercise 1.2): across the 672,443 classes with more than one curve there are 0 violations;
- parity: for semistable curves, Exercise 0.2 gives $w = (-1)^{\omega(N)+1}(-1)^{\#\text{non-split}}$, and w must equal $(-1)^r$. This holds for all 1,513,650 semistable curves (0 failures). It tests the reduction types, the split criterion at large primes and the dataset's ranks at once.

The audit. 8 quantities are never features, each with its reason recorded in code (`EXCLUDED`), and `feature_columns` asserts that no excluded name slips into a feature set:

excluded	why
rank	the label
analytic Sha	from the BSD formula, which needs r; > 1 for 20.4% of rank 0, 0.1% of rank 2
root number	$= (-1)^r$; used only as a sanity check in Section 4
regulator, $L^{(r)}(E,1)$	need the rank; $L(E,1) \neq 0 \iff r_{\text{an}} = 0$
label, class, index	identifiers; LMFDB orders class letters by the a_p, so letters are a coded feature

The signal, before any training. Figure 3 shows the average of $a_p/\sqrt{p}$ by rank. Rank-2 curves have strongly negative a_p, so more points mod p, exactly as Birch and Swinnerton-Dyer saw (Exercise 2.2). Rank 0 sits slightly above zero and rank 1 almost exactly on it, which is what the heuristic $-\sum a_p/p \approx (r-\frac{1}{2})\log\log X$ predicts. A fixed formula with no parameters, the Mestre–Nagao-style sum

$$S(E) = \sum_{p<100,\ p\nmid N} \frac{-a_p \log p}{p},$$

averages -1.29, 0.12 and 4.34 for ranks 0, 1 and 2. Used as a score on the complete range, it reaches an ROC-AUC of 0.965 on Task 2 and only 0.714 on Task 1. That reframes both tasks. Task 2 is not hopeless: a formula from 1965 already ranks rank-2 curves above rank-0 curves most of the time, so it is the baseline every learned model must beat. Task 1 is hard for the a_p because rank 0 and rank 1 look alike in Figure 3; what separates them is parity, which lives in the bad primes.

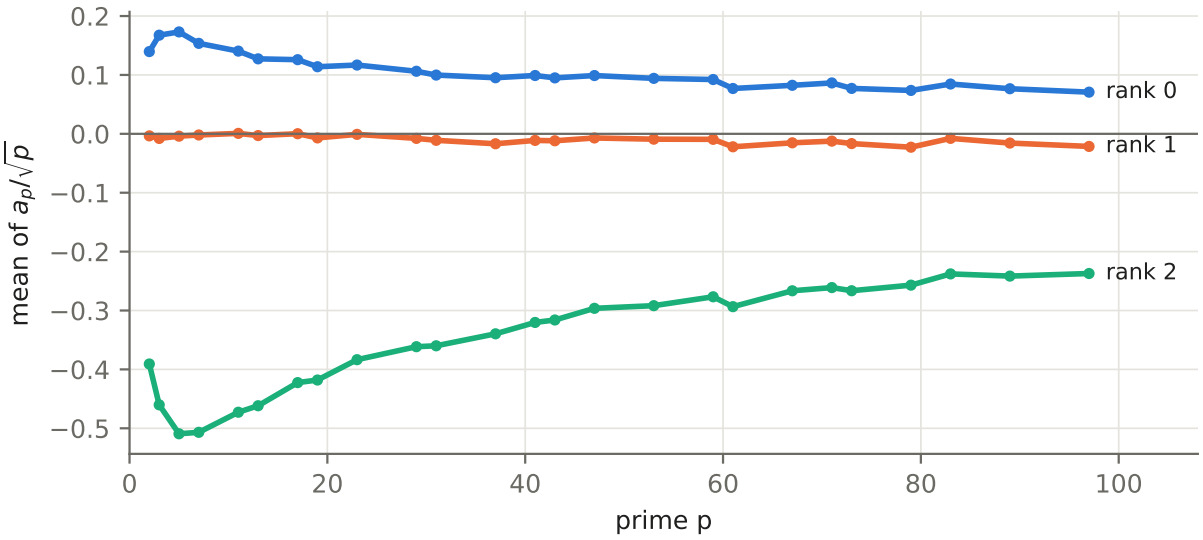


Figure 3: Mean of $a_p/\sqrt{p}$ over curves with good reduction at p and $N \le 500{,}000$, by rank.

What these features cannot do. The L-function's behaviour at $s=1$ depends on roughly $\sqrt{N}$ of its coefficients, and we give the model the a_p for $p<100$ only. At $N \approx 10^5$ that is a useful sample; at $N \approx 2\times 10^8$ it is a much smaller one. So we expect the signal in S to weaken as N grows, and Section 3 builds the splits that measure it.

Exercises.

Exercise 2.1. Let p be odd. Show that $y^2 + (a_1x + a_3)y = x^3 + a_2x^2 + a_4x + a_6$ has as many solutions in $\mathbb{F}_p^2$ as $Y^2 = 4x^3 + b_2x^2 + 2b_4x + b_6$, and deduce $a_p = -\sum_x \chi(g(x))$. Where does the argument fail for $p=2$, and how does the code handle $p=2$?

Exercise 2.2. Birch and Swinnerton-Dyer observed $\prod_{p\le X,\,p\nmid N} \#E(\mathbb{F}_p)/p \approx C(\log X)^r$. Take logarithms, use $\log(1+u)\approx u-u^2/2$, and assume a_p^2 averages to p (Sato–Tate) to show this predicts $\sum_{p\le X} -a_p/p \approx (r-\frac{1}{2})\log\log X + O(1)$. Explain Figure 3 with it: why is rank 0 above zero and rank 1 so close to it?

Exercise 2.3. Suppose the split criterion had the wrong sign, so that it reported split as non-split for every bad prime $p \geq 100$. Which curves would fail the parity check, and roughly how many would you expect? Why is a check that mixes several computations stronger than one that tests each alone?

Interview prep. *"Couldn't you just have used a library for the a_p?"* "The library didn't build on my machine, so I wrote a vectorized point count and validated it against facts it must satisfy: brute force, known modular forms, the Hasse bound, equality across 672 thousand isogeny classes, and the parity of the rank for 1.5 million semistable curves. Then, before training anything, I checked a formula from the 1960s. It already gets an AUC of 0.965 on my headline task, so that is the baseline my models have to beat."

3 Evaluation when examples aren't independent

A test score means something only if the test curves are new to the model in the way that matters. Two things break that here: isogenous curves are near-copies, and the dataset is two families with different base rates. Step 3 builds splits that handle both (`src/eclab/splits.py`).

How much a curve-level split leaks. Take the Task 2 curves of conductor at most 500,000 and use the a_p for $p < 100$ as an exact key. Consider a "model" that predicts a test curve's rank by finding a training curve with the same key. Under a random 80/20 split *of curves*, 44.2% of test curves have such a match, and copying its label is right 100.0% of the time. Under an 80/20 split *of isogeny classes*, 0.00% have a match. Among these curves, no two different classes share the whole key (0 collisions), so the a_p vector is a fingerprint of the isogeny class. A flexible model can memorize fingerprints, and a curve-level split would reward it for that.

Folds by hashing the class. Each class goes to one of five folds by a keyed hash of its name. The assignment does not depend on row order, survives adding new curves, and is identical on every machine:

From `src/eclab/splits.py`

```
def class_fold(iso_class: pd.Series, k: int = N_FOLDS) -> np.ndarray:
    h = pd.util.hash_array(iso_class.to_numpy(dtype=object), hash_key=HASH_KEY, categorize=True)
    return (h % np.uint64(k)).astype(np.int8)
```

Four splits, four questions. Every split keeps classes whole; conductor-based splits do so automatically, since a class shares one conductor. Counts are Task 2 curves; π is the share of rank 2.

split	curves (train → test)	train	test	π train	π test
random	$N \leq 500$k, folds 1–4 → fold 0	1,216,168	303,380	22.9%	22.9%
conductor	$N \leq 400$k → $(400\text{k}, 500\text{k}]$	1,230,432	289,116	22.3%	25.7%
prime	prime $N \leq 10^8 \to N > 2 \cdot 10^8$	156,144	98,600	39.4%	38.9%
family	$N \leq 500$k → prime $N > 500$k	1,519,548	362,296	22.9%	39.1%

The *prime* split is the clean test of size: train and test share a family and a base rate, and the gap between 10^8 and $2 \cdot 10^8$ keeps neighbouring conductors apart. The *family* split changes everything at once, including the base rate (from about 23% to 39%). If a model does worse there but not on the prime split, the cause is the family, not the size.

Which metric survives which shift. When only the base rate changes (label shift), ROC-AUC is unchanged: it depends only on how the scores of rank-2 curves compare with those of rank-0 curves, and the distribution of scores within each class does not move (Exercise 3.1). Accuracy, log-loss and calibration all move, even for a perfect model of the training distribution. So Sections 4–7 report ROC-AUC as the headline across splits, PR-AUC alongside it, and log-loss and calibration on the random split, using the correction of Exercise 1.3 when comparing across splits.

Cross-validation inside training. Model selection in Sections 4–6 uses the same folds within the training set, so tuning never sees the test set and never separates isogenous curves either.

What the splits cannot do. They decide which curves a model is tested on, not whether its score beats chance by more than noise. Every comparison from Section 4 on comes with a bootstrap interval over isogeny classes, which are the independent units, not curves.

Exercises.

Exercise 3.1. Show that ROC-AUC equals $\mathbb{P}(s(X_1) > s(X_0))$, where X_1 and X_0 are independent draws of a rank-2 and a rank-0 curve. Deduce that it is unchanged under label shift, and give an example of a covariate shift that changes it.

Exercise 3.2. Use the class-size counts from Section 1 to compute the prediction of Exercise 1.2 for the share of test curves with a twin in training. Compare it with the measured 44.2%. Why should they agree only roughly?

Exercise 3.3. Give two concrete ways a split made with `rng.permutation(len(df))` and a fixed seed can silently change between two runs of the same code. Why does hashing the class name avoid both?

Interview prep. *"How did you avoid leakage in your splits?"* "Isogenous curves share every a_p and the rank. With a curve-level split, 44% of test curves had an exact twin in training, and copying its label was 100% accurate. So I hash the isogeny class into folds. For distribution shift I separate two questions: a prime-to-prime split tests larger conductors within one family and base rate, and a cross-family split tests everything at once. AUC is my headline metric across splits because label shift does not move it."

4 Baselines and logistic regression

Step 4 fits the simplest models on both tasks and all four splits. It also builds what every later section reuses: the metrics, intervals over isogeny classes, and saved test predictions (`src/eclab/evaluate.py`, `src/eclab/models.py`).

The models. *Majority* predicts the training share π for every curve. *Logistic regression* models $\mathbb{P}(y = 1 \mid x) = \sigma(\beta_0 + \beta^\top x)$, with $\sigma(t) = 1/(1 + e^{-t})$ and each feature standardized, and chooses β to minimize the log-loss

$$L(\beta) = -\frac{1}{n} \sum_i [y_i \log q_i + (1 - y_i) \log(1 - q_i)], \qquad q_i = \sigma(\beta_0 + \beta^\top x_i).$$

L is convex, so the optimum is unique; with 10^6 curves and 40 features the ridge penalty barely matters. We fit it on three feature sets (invariants, a_p, both) and on S alone. A logistic curve is increasing in S, so "S only" has exactly the AUC of the formula, but now it also gives probabilities.

Metrics. ROC-AUC is the headline (Section 3). PR-AUC (average precision) is the area under the precision–recall curve; a model that ignores its input scores π, so it measures how well the rare class is found. On Task 2 (random split) majority scores PR-AUC 0.229 and log-loss 0.538, which is $H(\pi)$ as in Exercise 0.3.

Intervals over classes. Each bootstrap replicate draws isogeny classes with replacement and gives every curve its class's draw count w_i as a weight. Every model is scored on the same 200 replicates, so the difference between two models gets a *paired* interval, much narrower than comparing two separate intervals. Sorting the scores once makes each replicate linear in n:

From src/eclab/evaluate.py

```
def __call__(self, w):                      # self.group: ties share a group, in score order
    w = w[self.order]
    pos = np.bincount(self.group, weights=w * self.y)
    neg = np.bincount(self.group, weights=w * (1 - self.y))
    below = np.cumsum(neg) - neg            # negative weight strictly below each group
    return (pos * (below + neg / 2)).sum() / (pos.sum() * neg.sum())
```

Task 2. ROC-AUC by split, with the paired difference from S in parentheses. Every difference's 95% interval excludes zero, and every AUC interval is narrower than ± 0.004 (Figure 5).

model	random	conductor	prime	family
majority	0.500	0.500	0.500	0.500
S only	0.965	0.956	0.792	0.815
invariants	0.700 (-0.266)	0.675 (-0.282)	0.552 (-0.240)	0.544 (-0.271)
a_p	0.975 (+0.010)	0.967 (+0.011)	0.812 (+0.021)	0.827 (+0.012)
invariants + a_p	0.991 (+0.025)	0.980 (+0.024)	0.818 (+0.026)	0.819 (+0.003)

Four things stand out.

1. Learning helps, a little. Logistic regression on the a_p beats the fixed formula, and adding the invariants gives 0.991 on the random split ($+0.0252$, interval $[+0.0246, +0.0258]$). Log-loss falls from 0.201 to 0.104 and PR-AUC rises from 0.876 to 0.961.
2. The learned weights *are* S, nearly. Figure 4 puts the logistic weights on $-a_p$ next to $\log p/p$: the correlation is 0.998. The difference is the decay: for $p \geq 7$ the learned weight falls like $p^{-1.07}$, faster than $\log p/p$. Exercise 4.2 explains why $1/p$ is the right weight for a linear score.
3. Larger conductors cost 0.15 to 0.17 of AUC, for every model, including S, which has no parameters to overfit. So the drop is not overfitting: the a_p for $p < 100$ simply say less when $N \approx 10^8$, as Section 2 predicted. The prime split, which holds the family and base rate fixed, loses as much as the family split, so size explains most of the drop.
4. The invariants alone are weak (about 0.70 on the random split, near chance on large conductors). Whatever they add, they add next to the a_p.

Task 1 and the parity trap.

model	random	conductor	prime	family
majority	0.500	0.500	0.500	0.500
S only	0.716	0.686	0.667	0.679
invariants	0.579 (-0.137)	0.575 (-0.111)	0.874 (+0.207)	0.386 (-0.293)
a_p	0.710 (-0.005)	0.680 (-0.006)	0.679 (+0.012)	0.682 (+0.003)
invariants + a_p	0.725 (+0.009)	0.697 (+0.011)	0.950 (+0.282)	0.642 (-0.037)

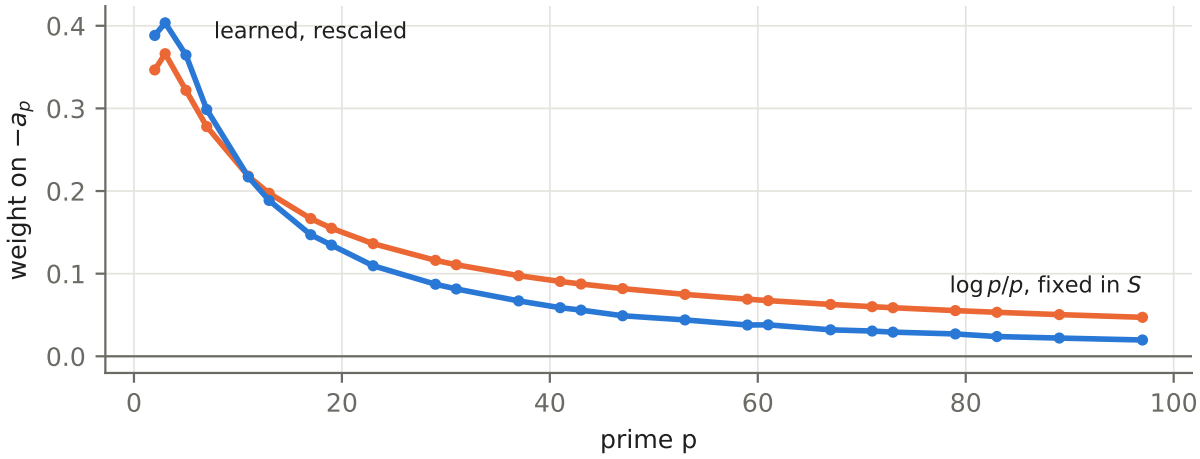


Figure 4: Logistic weights on $-a_p$ (Task 2, random split, per unit of a_p), rescaled to match $\log p/p$, the weight in S.

On the random split logistic regression barely beats S (0.725). The sanity check promised in Section 2 explains why the task looked easy: the root number alone scores 0.908, and adding it to the features gives 0.998. Most of what is learnable in Task 1 is parity. The invariants contain parity for every semistable curve (Exercise 0.2), but as an XOR of reduction counts, and a linear model cannot compute an XOR (Exercise 4.1). Two cells show this from both sides:

- *Prime split, invariants: 0.874.* When N is prime, $w = (-1)^{\#\text{non-split}}$, which *is* linear in one feature, and a model trained on prime conductors finds it.
- *Family split, invariants: 0.386, worse than chance.* In the complete range n_{split} alone has AUC 0.521, almost no signal, and the model gives it the small weight $+0.062$ for other reasons. On prime conductors, split reduction means $w = +1$, so even rank, and n_{split} as a score has AUC 0.142. A harmless weight became decisive with the wrong sign.

AUC hides a calibration failure. On the family split the Task 2 logistic model has log-loss 2.60, against 0.104 on the random split, while its AUC looks fine. Training conductors stop at $\log N = 13.1$ and test conductors reach 19.5; a logit is linear in the features, so it extrapolates into confident, wrong probabilities. Section 6 returns to calibration.

What logistic regression cannot do. It adds up evidence feature by feature. Parity is an interaction, and so is "this a_p is small *because* $p \mid N$". Section 5 uses trees, which split on one feature and then another.

Exercises.

Exercise 4.1. Let N be squarefree with exactly two prime factors. Using Exercise 0.2, list the parity of the rank for $(n_{\text{split}}, n_{\text{non-split}}) = (2,0), (1,1), (0,2)$, and show that no function $\beta_0 + \beta_1 n_{\text{split}} + \beta_2 n_{\text{non-split}}$ has the right sign on all three. What happens when N is prime?

Exercise 4.2. Suppose that for good p the a_p are independent with variance p (Sato–Tate) and that the mean of a_p is μ_p for rank-0 curves and $\mu_p - \delta_p$ for rank-2 curves. The linear score with the

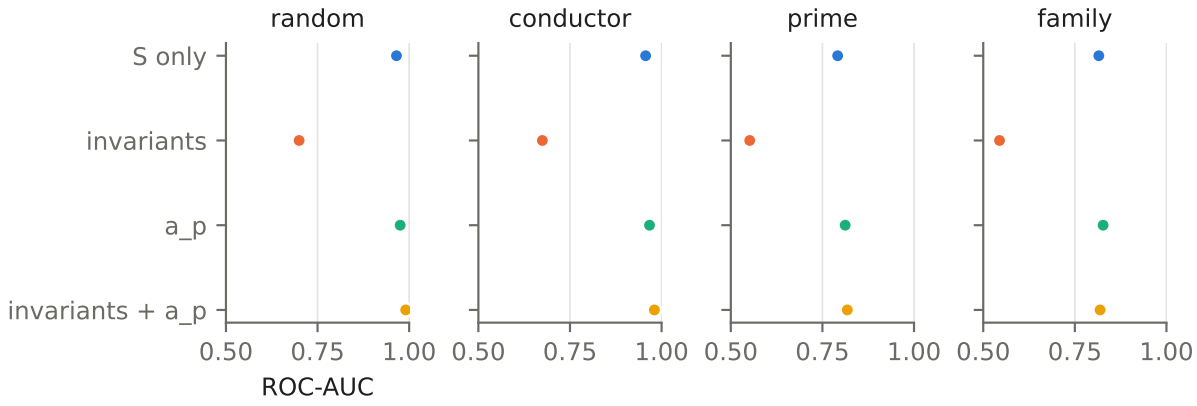


Figure 5: Task 2 ROC-AUC with 95% class-bootstrap intervals (narrower than the dots).

best separation weights a_p by δ_p/p. Use Exercise 2.2 to argue that δ_p is roughly constant in p, and compare the predicted weight with the measured $p^{-1.07}$.

Exercise 4.3. Show that the AUC computed with integer weights w_i equals the AUC of the data set in which curve i appears w_i times. If every class had exactly k curves with identical scores, by roughly what factor would a bootstrap over curves understate the standard error?

Interview prep. *"0.99 AUC, so what's the baseline?"* "A formula from the 1960s with no parameters gets 0.965. Logistic regression learns almost the same weights, and adds 0.025 with the invariants; I report that difference with a paired bootstrap over isogeny classes. The more important number is the drop to about 0.82 on large conductors, which hits the formula equally, so it is about information, not overfitting. And on the easy task, the model mostly learned parity, which I confirmed by adding the root number and getting 0.998."

5 Gradient boosting, and what each feature group adds

Logistic regression adds evidence one feature at a time. Step 5 replaces it with trees, which can combine features, and asks two ablation questions on every split: what do the invariants add to the a_p, and what do trees add to a linear model?

Boosted trees. A regression tree splits the feature space by thresholds ($a_7 \le -2$? then $\log N \le 11.4$?) and predicts a constant in each leaf. Boosting builds a score $F = \sum_m \nu h_m$ one tree at a time: tree h_m is fit to the negative gradient of the log-loss at the current score, which for curve i is $y_i - \sigma(F_i)$, the part of the label not yet explained (Exercise 5.2). The step size $\nu = 0.1$ keeps each tree's contribution small. scikit-learn's histogram version bins each feature into at most 255 values first, so a tree on 1.2 million curves takes a fraction of a second.

Stopping on whole classes. Boosting stops when the log-loss on held-out data stops improving. scikit-learn's built-in version holds out *random rows*, which would put isogenous twins on both sides, exactly the leak of Section 3. So the held-out rows are fold 1 of the class hash:

From `src/eclab/models.py`

```
def fit(self, train, y, val):           # val = splits.validation_mask(train): whole classes
    X = train[self.columns].to_numpy(np.float32)
    self.model.fit(X[~val], y[~val], X_val=X[val], y_val=y[val])
```

The only tuned setting is the number of leaves per tree, chosen by validation log-loss on Task 2: 0.0561, 0.0499 and 0.0558 for 15, 63 and 255 leaves. We use 63 everywhere. It stopped at the cap of 2000 trees while still improving slowly, so the boosting numbers are, if anything, slightly pessimistic.

Task 2. ROC-AUC by split; the last two rows are paired differences with half the width of their 95% interval.

model	random	conductor	prime	family
S only	0.965	0.956	0.792	0.815
logistic, inv + a_p	0.991	0.980	0.818	0.819
boosting, invariants	0.741	0.705	0.516	0.541
boosting, a_p	0.982	0.974	0.823	0.828
boosting, inv + a_p	0.998	0.992	0.825	0.838
invariants add	+0.0159 ±0.0004	+0.0179 ±0.0004	+0.0023 ±0.0012	+0.0102 ±0.0005
trees add	+0.0071 ±0.0003	+0.0120 ±0.0004	+0.0074 ±0.0014	+0.0191 ±0.0008

"Invariants add" is boosting on both groups minus boosting on the a_p; "trees add" is boosting minus logistic regression, both on both groups.

1. *The complete range is close to solved.* Boosting reaches 0.998 on the random split, with log-loss 0.050 and PR-AUC 0.992. A clearer way to say it: $1 - \text{AUC}$ is the share of (rank 2, rank 0) pairs put in the wrong order. It falls from 0.0346 for S to 0.0023, about 15 times fewer.
2. *The invariants help through* $\log N$. In the permutation importances (Figure 7) the small-p a_p dominate, as the logistic weights did, and the only invariant in the top twelve is $\log N$ (rank 5). The reason is that the meaning of S changes with N. Within the complete range, the median S of rank-2 curves falls from 6.07 for $N \le 58,290$ to 3.68 for $N > 434,280$, and rank 0 moves from -1.98 to -1.06, so the AUC of S falls from 0.991 to 0.956. The same S is strong evidence for rank 2 at large N and weak evidence at small N. A tree learns this as a threshold that depends on N; a sum of separate terms cannot.
3. *That help does not extrapolate.* On the prime split, where every test conductor is larger than every training conductor, the invariants add only +0.002. A tree is constant beyond its largest threshold, so for $N > 10^8$ it uses the rule learned at $N \approx 10^8$.
4. *Large conductors stay hard.* Boosting gets 0.825 and 0.838 on the prime and family splits: better than S, still far from the complete range. Every model has the same ceiling there, which again points to information, not modelling.

Task 1: trees learn parity. Boosting on the invariants *alone* scores 0.768 on Task 1, more than logistic regression on everything (0.725). On semistable test curves, where parity is a function of the reduction counts, it scores 0.945 against 0.763: an XOR takes a few splits of a small tree (Exercise 5.1). With the a_p added, boosting reaches 0.896. This is the parity trap from Section 0 coming true: on Task 1 a stronger model looks better largely by rediscovering the root number, which is why Task 2 is the headline.

Reading importances carefully. Shuffling one feature measures what the model loses without it, *given the others*. Correlated features share credit (Exercise 5.3), so a small importance does not mean a feature carries no information, only that the model can do without it.

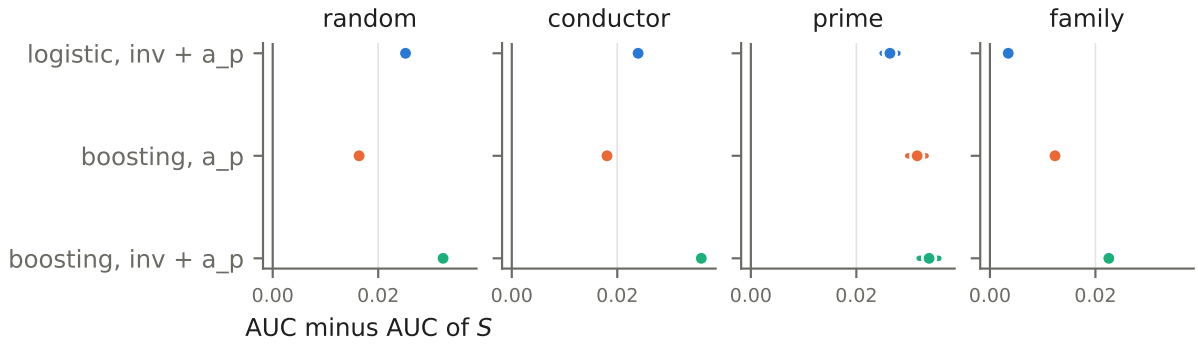


Figure 6: Task 2: ROC-AUC minus that of S, with paired 95% class-bootstrap intervals (narrower than the dots).

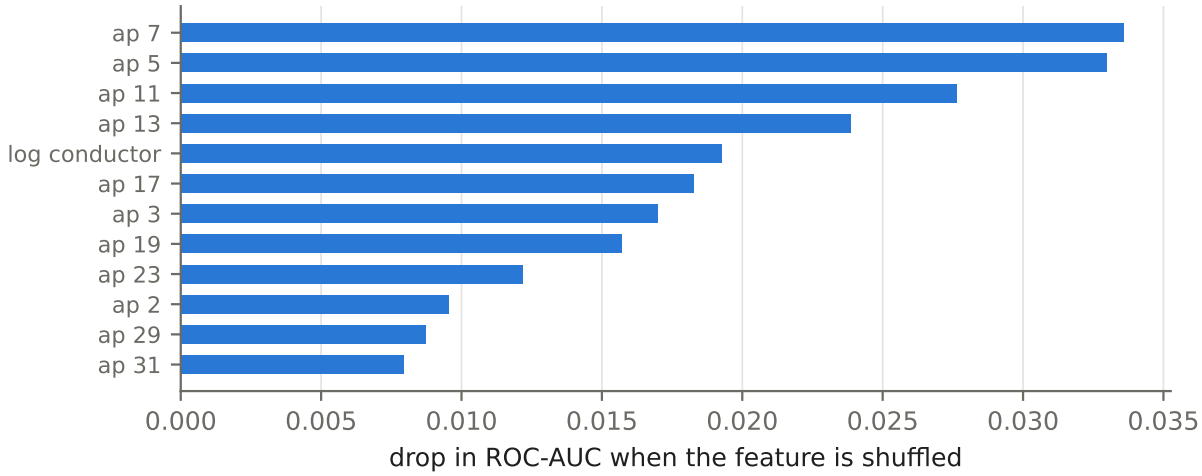


Figure 7: Permutation importance (Task 2, random split, 200,000 test curves): the drop in ROC-AUC when one feature's values are shuffled.

What boosting cannot do. Trees do not extrapolate, which matters on exactly the splits where we most want to generalize. They also treat a_5 and a_7 as unrelated columns, with no notion that both are traces of Frobenius on the same object, scaled by $\sqrt{p}$. Section 6 tries a network, which uses smooth functions of all the a_p at once, and checks whether any of these probabilities can be trusted.

Exercises.

Exercise 5.1. Using Exercise 4.1, draw a tree of depth 3 on $(n_{\text{bad}}, n_{\text{non-split}})$ that gives the parity of the rank exactly for semistable curves with one or two bad primes. How many leaves does a tree need for up to k bad primes, and why does boosting with 63 leaves per tree have no trouble?

Exercise 5.2. For the log-loss $\ell(y, F) = -y \log \sigma(F) - (1-y) \log(1-\sigma(F))$, show that $\partial \ell / \partial F = \sigma(F) - y$ and $\partial^2 \ell / \partial F^2 = \sigma(F)(1-\sigma(F))$. Deduce the Newton value for a leaf containing curves $i \in I$: $\sum_I (y_i - q_i) / \sum_I q_i(1-q_i)$.

Exercise 5.3. Suppose we added an exact copy of the column a_7. What would happen to the permutation importance of each copy, and why? What does this say about reading Figure 7 as a ranking of which primes "matter" for the rank?

Interview prep. *"What did gradient boosting buy you?"* "On the complete range, 15 times fewer misordered pairs than the classical formula, AUC 0.998. The ablation shows where it comes from: the formula's meaning drifts with the conductor, and trees learn a conductor-dependent threshold. That same mechanism can't extrapolate, so on larger conductors boosting gains little. On the parity task it learned the root number from reduction types, which is exactly why I don't headline that task."

6 A small neural network, and whether to trust the probabilities

Step 6 asks two questions. Does a neural network, which uses smooth functions of all features at once, beat the trees? And are any of these models' probabilities right, in the sense that among curves given 0.3, about 30% have rank 2?

The network. Two hidden layers of 128 and 64 ReLU units, $h \mapsto \max(0, Wh + b)$, then a sigmoid; inputs standardized; the log-loss plus a small L^2 penalty, minimized by Adam on minibatches of 1024 curves. On both feature groups it has 13,569 weights. As with boosting, scikit-learn's own early stopping would hold out random rows, so we train one epoch at a time and keep the epoch with the best log-loss on the validation classes:

From src/eclab/models.py

```
for _ in range(self.max_epochs):
    self.net.partial_fit(Xt, yt, classes=[0, 1])    # one pass over shuffled minibatches
    loss = log_loss(yv, self.net.predict_proba(Xv)[:, 1])
    if loss < best - 1e-5:
        best, best_net, wait = loss, copy.deepcopy(self.net), 0
    else:
        wait += 1
        if wait >= self.patience:
            break
```

On the random split the best epoch was 12 of 15; an epoch takes about 7 seconds on a laptop CPU.

Network against trees. Paired differences in Task 2 ROC-AUC, network minus boosting on the same features, with half the width of the 95% class-bootstrap interval:

features	random	conductor	prime	family
a_p only	-0.0016 ± 0.0003	-0.0025 ± 0.0003	$+0.0031$ ± 0.0009	$+0.0004$ ± 0.0004
invariants + a_p	$+0.0007$ ± 0.0001	$+0.0019$ ± 0.0002	$+0.0050$ ± 0.0012	-0.1673 ± 0.0020

Inside the training range it is a tie in every practical sense. The network reaches 0.998 on the random split, ahead of boosting by $+0.0007$: an interval that excludes zero, on a difference no one would notice. On the a_p alone it is slightly *behind*. This is the lesson of the calc project again. A more flexible model is not automatically a better one, and the honest report is "tie", with the interval to show it.

Then it falls apart. On the family split the network on invariants and a_p scores 0.671, against 0.838 for boosting, while the network on the a_p alone is fine (0.828). The culprit is extrapolation. A ReLU network is piecewise linear, so beyond the training data it continues along its last linear piece (Exercise 6.1); a tree stays constant. The test conductors run from $\log N \approx 13$, the edge of training, to 19.5. Split the family test set into fifths by $\log N$. From the lowest fifth ($\log N \le 17.5$) to the highest ($\log N > 19.2$), boosting slides from 0.888 to 0.811, and the network falls from 0.798 to 0.598. The a_p have the same bounded range at every conductor (Hasse), so a network that sees only them has nothing to extrapolate. The prime split does not show this, because there the training data already reach $N = 10^8$. The remedy is not a better network. It is to give a model only features whose range does not move between training and use, or to say clearly which inputs it may not be trusted on.

Calibration. A model is *calibrated* if $\mathbb{P}(y = 1 \mid q(x) = t) = t$ for every t. The *reliability diagram* bins the predictions and plots the observed share of rank 2 in each bin against the mean prediction; the *expected calibration error* (ECE) is the average gap, weighted by bin size. On the random split:

model	ROC-AUC	log-loss	ECE
logistic on S	0.9654	0.2007	0.0168
logistic, inv + a_p	0.9906	0.1044	0.0066
boosting, inv + a_p	0.9977	0.0504	0.0016
network, inv + a_p	0.9984	0.0445	0.0045

All four are well calibrated where they were trained (Figure 8, left). Log-loss rewards calibration and ranking together, and there the network and boosting tie again. The network under-predicts slightly in the middle, which temperature scaling could fix without changing its AUC (Exercise 6.3).

Calibration under shift. On the family split the base rate jumps from 22.9% to 39.1%. Section 1's correction (Exercise 1.3) needs the new base rate, which we would not know in practice. It can be estimated from unlabelled test curves by EM (Saerens et al., 2002): shift the predictions to a guessed rate, average them, repeat (Exercise 6.2). Boosting's raw predictions average 35.2%; EM estimates 38.2%, close to the truth. But the calibration does not improve: ECE goes from 0.175 to 0.178, the same as with the true rate (0.178). The right panel of Figure 8 shows why. The curve is *flat*, not shifted: curves given 0.1 are about 37% rank 2 and curves given 0.9 about 56%. The model is overconfident because the a_p carry less information at large N, and that is covariate shift, not label shift. Correcting the base rate fixes the average, not the slope. Fixing the slope needs some labelled curves from the new family.

What these models cannot do. On the complete range three model families agree to the third decimal, so the remaining errors are probably not a modelling problem. Off it, every model hits the same ceiling near 0.83, and the models that tried to use $\log N$ to get past it were hurt by it. Section 7 looks at what the a_p themselves do as N grows, through the murmurations of He, Lee, Oliver and Pozdnyakov.

Exercises.

Exercise 6.1. Show that a network built from affine maps and $\max(0, \cdot)$ is continuous and piecewise linear, and that along any ray $x_0 + t\,v$ its pre-sigmoid output is linear in t for t large enough. Why does this make the network's predictions go to 0 or 1 as $\log N$ grows, while a tree's stay fixed?

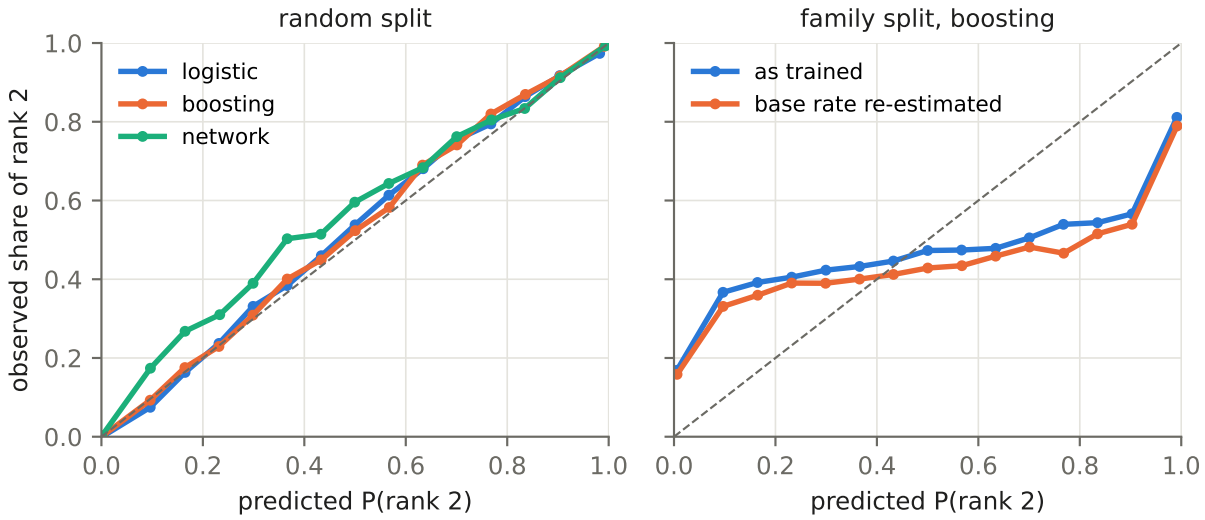


Figure 8: Reliability diagrams for Task 2 (bins with at least 200 curves). Left: random split. Right: boosting on the family split, before and after re-estimating the base rate by EM.

Exercise 6.2. Under label shift, suppose $q(x)$ is calibrated for training base rate π, and let $q_{\pi'}$ be q shifted to π' as in Exercise 1.3. Show that the true test rate π^* satisfies $\pi^* = \mathbb{E}_{\text{test}}[q_{\pi^*}(x)]$, so it is a fixed point of the EM iteration. Why does this argument fail on the family split?

Exercise 6.3. *Temperature scaling* replaces a logit z by z/T. Show that this does not change the ROC-AUC. If the log-loss on held-out curves is $L(T)$, show that L is convex in $1/T$, and say which way T should move for a model that is overconfident.

Interview prep. *"Did deep learning help?"* "I tried a small network with the same isogeny-safe early stopping. Inside the training range it tied gradient boosting: 0.998 both, a difference of 0.0007. Outside it, the network fell to 0.67 while trees kept 0.84, because a ReLU network extrapolates linearly in the conductor and a tree stays flat. I also checked calibration: all models are calibrated in-distribution. Under the shift, re-estimating the base rate by EM got the rate right, 38% against 39%, but didn't fix calibration, because the real problem was overconfidence, not the base rate."

Solutions to exercises

Exercise 0.1. The gamma factor is nonzero at $s = 1$ ($\Gamma(1) = 1$), so $\operatorname{ord}_{s=1}\Lambda = r_{\text{an}}$ and $\Lambda(E,s) = c\,(s-1)^r + O((s-1)^{r+1})$ with $c \neq 0$ and $r = r_{\text{an}}$. Substituting $s \mapsto 2 - s$ gives $\Lambda(E, 2-s) = c\,(1-s)^r + \cdots = (-1)^r c\,(s-1)^r + \ldots$. Comparing leading coefficients in (1) gives $c = w(-1)^r c$, so $w = (-1)^r$.

Exercise 0.2. With $w = -\prod_{p|N} w_p$ and $w_p = -a_p$ for every $p \mid N$, $w = -\prod_{p|N}(-a_p) = -(-1)^{\omega(N)}\prod a_p = (-1)^{\omega(N)+1}\prod a_p$. For `11.a`: $\omega = 1$, $a_{11} = 1$, so $w = +1$, consistent with rank 0. For `37.a`: $\omega = 1$, $a_{37} = -1$, so $w = -1$, consistent with rank 1.

Exercise 0.3. A constant prediction of class 1 is right on a fraction π of curves and class 0 on $1-\pi$; take the larger. A constant probability q has expected log-loss $-\pi \log q - (1-\pi)\log(1-q)$, minimized at $q = \pi$ with value $H(\pi)$. When π is small, predicting "rank 0" always scores $1-\pi$, close to 1, while finding no rank-2 curve at all. Log-loss rewards confidence that is justified, and PR-AUC measures how well the model ranks the rare class, so both expose that predictor.

Exercise 1.1. φ restricts to a homomorphism $E(\mathbb{Q}) \to E'(\mathbb{Q})$ with finite kernel, so $\operatorname{rk} E(\mathbb{Q}) \leq \operatorname{rk} E'(\mathbb{Q})$. The dual isogeny $\hat{\varphi} : E' \to E$ is also defined over $\mathbb{Q}$ with finite kernel, giving the reverse inequality.

Exercise 1.2. For good p, φ reduces to an isogeny $\tilde{E} \to \tilde{E}'$ over $\mathbb{F}_p$ that commutes with Frobenius π. Then $\#\tilde{E}(\mathbb{F}_p) = \deg(1-\pi)$ on both sides, since $1-\pi$ has the same characteristic polynomial on the Tate modules, which the isogeny identifies after tensoring with $\mathbb{Q}_\ell$. With random curve-level splitting, a test curve in a class of size k has an untouched twin unless all $k-1$ others also went to test, so the fraction is $\sum_k f_k\,(1 - 0.2^{k-1})$, where f_k is the share of curves in classes of size k. It underestimates the damage because a flexible model memorizes the exact a_p vector, so one twin is enough to recall the label.

Exercise 1.3. By Bayes, the posterior odds are the likelihood ratio $p(x \mid r=2)/p(x \mid r=0)$ times the prior odds. Under label shift the likelihood ratio is unchanged, so the new odds are the old odds divided by $\pi/(1-\pi)$ and multiplied by $\pi'/(1-\pi')$. With the numbers from this section, the factor is about $(0.391/0.609)/(0.229/0.771) \approx 2.2$: the model should more than double its odds of rank 2 on the large prime-conductor curves.

Exercise 2.1. For odd p the substitution $Y = 2y + a_1x + a_3$ is a bijection of $\mathbb{F}_p$ for each fixed x, and $Y^2 = 4(x^3 + a_2x^2 + a_4x + a_6) + (a_1x + a_3)^2 = g(x)$. For each x, $Y^2 = g(x)$ has $1 + \chi(g(x))$ solutions, so there are $p + \sum_x \chi(g(x))$ affine points. Adding the point at infinity, $a_p = p + 1 - \#E(\mathbb{F}_p) = -\sum_x \chi(g(x))$. For $p = 2$ one cannot divide by 2, so the code counts the four pairs $(x, y) \in \mathbb{F}_2^2$ directly.

Exercise 2.2. With $u = (1-a_p)/p$, $\#E(\mathbb{F}_p)/p = 1 + u$ and $\log(1+u) \approx u - u^2/2$. The first-order term gives $\sum 1/p - \sum a_p/p$. In the second, $u^2 \approx a_p^2/p^2$, which averages to $1/p$, so it contributes $-\frac{1}{2}\sum 1/p$. By Mertens, $\sum_{p \leq X} 1/p = \log\log X + O(1)$, so

$$\log \prod \approx \tfrac{1}{2} \log\log X - \sum a_p/p.$$

Setting this equal to $r \log\log X + O(1)$ gives $-\sum a_p/p \approx (r - \frac{1}{2}) \log\log X$. So rank 0 has a positive bias in the a_p, rank 1 a small negative one, and rank 2 a large negative one: the order in Figure 3, where ranks 0 and 1 sit on either side of zero, close together. (Goldfeld made this precise in 1982, conditionally on hypotheses about $L(E, s)$.)

Exercise 2.3. Every semistable curve with an odd number of bad primes $p \geq 100$ would get the wrong w, so the check would fail on a large fraction of curves of prime conductor above 100, roughly everything in the prime family. One check that combines the reduction types, the split criterion and the ranks can only pass if all three are right (or their errors cancel exactly), so passing it on 1.5 million curves is strong evidence for all three.

Exercise 3.1. The ROC curve integrates the true-positive rate against the false-positive rate. Its area is $\mathbb{P}(s(X_1) > s(X_0))$, by writing both rates as tail probabilities and integrating. This probability depends only on the score distributions given each class. Label shift changes the class proportions, not these distributions, so AUC is unchanged. A covariate shift that changes it: on larger conductors the a_p for $p < 100$ carry less information, so the rank-2 and rank-0 score distributions overlap more, and AUC falls.

Exercise 3.2. The prediction is $\sum_k f_k(1 - 0.2^{k-1})$, from the class sizes of the Task 2 curves. It should agree only roughly with the measured 44.2%: the measurement uses Task 2 curves only, and an exact a_p key can match a curve in another class in principle (here it never does).

Exercise 3.3. Adding or removing a single row shifts the permutation of every later row; and reading the file in a different order (a different source, or a pandas sort) changes which curve gets which random number. The hash depends only on the class name and a fixed key, so neither matters.

Exercise 4.1. With $a_p = +1$ for split and -1 for non-split primes, $\prod a_p = (-1)^{n_{\text{non-split}}}$, so $w = (-1)^{\omega(N)+1+n_{\text{non-split}}}$. For $\omega = 2$: $(2,0)$ gives $w = -1$ (odd rank), $(1,1)$ gives $w = +1$ (even) and $(0,2)$ gives $w = -1$ (odd). Since $n_{\text{split}} = 2 - n_{\text{non-split}}$, any linear score is an affine function of $n_{\text{non-split}} \in \{0,1,2\}$; one that is positive at 0 and 2 is positive at 1. When N is prime, $w = (-1)^{n_{\text{non-split}}}$: odd rank exactly when $n_{\text{non-split}} = 1$, which a single weight captures.

Exercise 4.2. For two classes with means m_0, m_1 and common covariance Σ, the best linear separation (Fisher) uses weights $\Sigma^{-1}(m_1 - m_0)$, here $-\delta_p/p$ on a_p. By Exercise 2.2 the expected $-\sum_{p \le X} a_p/p$ differs between rank 2 and rank 0 by about $2 \log\log X \approx 2\sum_{p \le X} 1/p$, which is what a constant $\delta_p \approx 2$ gives. So the weight should fall like $1/p$, close to the measured $p^{-1.07}$ and faster than $\log p/p$. (The $\log p$ in Mestre's sum comes from L'/L in the explicit formula; it is a good weight, not the best linear one for this task.)

Exercise 4.3. The weighted AUC is $\sum_{i \in +} \sum_{j \in -} w_i w_j ([s_i > s_j] + \frac{1}{2}[s_i = s_j])/(W_+ W_-)$, which counts each pair of copies once in the repeated data set. If every class has k identical curves, the data hold only n/k independent units, so the true standard error is about $\sqrt{k}$ times what a bootstrap over the n curves reports.

Exercise 5.1. Split first on $n_{\text{bad}} \le 1$. If yes (N prime), split on $n_{\text{non-split}} \le 0$: even rank if yes, odd if no. If no ($\omega = 2$), split on $n_{\text{non-split}} \le 0$ (odd), then on $n_{\text{non-split}} \le 1$ (even if yes, odd if no). One leaf per cell $(\omega, n_{\text{non-split}})$ with $0 \le n_{\text{non-split}} \le \omega$ always suffices, which is $\sum_{\omega=1}^{k}(\omega+1) = k(k+3)/2$ leaves. In the complete range $\omega \le 6$ (since $2 \cdot 3 \cdot 5 \cdot 7 \cdot 11 \cdot 13 \cdot 17 > 500{,}000$), so 27 leaves are enough, and boosting adds up many trees, none of which has to be exact.

Exercise 5.2. With $\sigma' = \sigma(1-\sigma)$, $\partial \ell/\partial F = -y(1-\sigma) + (1-y)\sigma = \sigma - y$, and differentiating again gives $\sigma(1-\sigma)$. Adding a constant c to every F_i in the leaf changes the loss by about $\sum_I (q_i - y_i)c + \frac{1}{2}\sum_I q_i(1-q_i)c^2$, which is smallest at $c = \sum_I (y_i - q_i)/\sum_I q_i(1-q_i)$.

Exercise 5.3. Shuffling one copy leaves the other intact, and the model can route its splits through either, so each copy can show a small importance even though a_7 itself matters; together they may show much less than one column did alone. Importance measures how much *this model* depends on

a column, given the others. It is not a ranking of how much information each a_p carries about the rank; for that, compare models refit without a feature, as the ablation does for whole groups.

Exercise 6.1. Affine maps and $\max(0, \cdot)$ are continuous and piecewise linear, and so are their compositions, with finitely many pieces. Along a ray each unit changes slope at most at finitely many t, so beyond the last such t the output is $z(t) = at + b$. If $a \neq 0$, $z \to \pm\infty$ and $\sigma(z) \to 0$ or 1: the further $\log N$ goes past the training range, the more confident the network becomes. A tree has finitely many thresholds; beyond the largest one in the direction v every tree returns the same leaf, so the prediction is constant.

Exercise 6.2. Under label shift, $q_{\pi^*}(x) = \mathbb{P}_{\text{test}}(y = 1 \mid x)$ by Exercise 1.3, so $\mathbb{E}_{\text{test}}[q_{\pi^*}(x)] = \mathbb{P}_{\text{test}}(y = 1) = \pi^*$. On the family split $p(x \mid y)$ also changes (the a_p separate the ranks less well at large N), so q_{π^*} is not the test posterior. EM still finds a rate close to π^* here, but the shifted probabilities are not calibrated.

Exercise 6.3. $z \mapsto z/T$ is increasing for $T > 0$, so the order of the scores, and hence the AUC, is unchanged. With $\beta = 1/T$, each term $-y \log \sigma(\beta z) - (1 - y) \log(1 - \sigma(\beta z))$ is convex in β: its second derivative is $z^2 \sigma(\beta z)(1 - \sigma(\beta z)) \geq 0$. An overconfident model has logits that are too large in absolute value, so the best T is greater than 1.